

# **Teoria da Decisão**

**Da construção de soluções às metaheurísticas: Representação, busca local e VNS**

Profa. Luiza Bernardes Real

luizabernardesreal@gmail.com

Universidade Federal de Minas Gerais  
Escola de Engenharia  
Graduação em Engenharia de Sistemas

## De onde partimos?

Na Aula 01, vimos a forma de um problema de otimização:

$$\begin{array}{ll}\min_{\mathbf{x}} & f(\mathbf{x}) \\ \text{sujeito a} & g_i(\mathbf{x}) \leq 0, \quad i = 1, \dots, p, \\ & h_j(\mathbf{x}) = 0, \quad j = 1, \dots, q, \\ & \mathbf{x} \in \mathcal{X}.\end{array}$$

### Questão desta aula

Como uma decisão simples cresce quando incorporamos, uma de cada vez, as condições relevantes de um problema real?

## Situação inicial: abastecer uma fábrica

Uma fábrica precisa receber 100 toneladas de matéria-prima.

Cada caminhão:

- transporta 10 toneladas;
- custa R\$ 800 por viagem;
- transporta uma carga indivisível.

### Decisão

Quantos caminhões devem ser contratados para atender à quantidade necessária ao menor custo?

## Degrau 1: quantos caminhões?

### Variável de decisão

$x$  = número de caminhões contratados.

### Modelo

$$\begin{array}{ll}\min_x & 800x \\ \text{sujeito a} & 10x \geq 100, \\ & x \in \mathbb{Z}_+.\end{array}$$

A solução é imediata:  $x^* = 10$ . Mas o modelo assume que todos os caminhões e todas as cargas são iguais.

## Degrau 2: diferentes tipos de caminhão

Agora existem três tipos de caminhão, com capacidades, custos e disponibilidades diferentes.

| Tipo | Capacidade | Custo por viagem | Disponibilidade |
|------|------------|------------------|-----------------|
| A    | 10 t       | R\$ 800          | 4 caminhões     |
| B    | 15 t       | R\$ 1.050        | 5 caminhões     |
| C    | 25 t       | R\$ 1.900        | 2 caminhões     |

### Nova decisão

Além de decidir quantos caminhões usar, precisamos decidir quantos caminhões de cada tipo serão contratados.

## Modelo com frota limitada

Seja

$x_i$  = número de caminhões do tipo  $i$ .

$$\begin{array}{ll}\min_{\mathbf{x}} & \sum_i c_i x_i \\ \text{sujeito a} & \sum_i q_i x_i \geq Q, \\ & 0 \leq x_i \leq d_i, \quad \forall i, \\ & x_i \in \mathbb{Z}, \quad \forall i.\end{array}$$

- $q_i$ : capacidade de um caminhão do tipo  $i$ ;
- $c_i$ : custo de um caminhão do tipo  $i$ ;
- $d_i$ : quantidade disponível de caminhões do tipo  $i$ ;
- $Q$ : quantidade total necessária.

## Uma regra gulosa

Como as capacidades são diferentes, comparamos o **custo por tonelada**.

| Tipo | Cap. | Custo     | R\$/t  |
|------|------|-----------|--------|
| A    | 10 t | R\$ 800   | R\$ 80 |
| B    | 15 t | R\$ 1.050 | R\$ 70 |
| C    | 25 t | R\$ 1.900 | R\$ 76 |

## Uma regra gulosa

Como as capacidades são diferentes, comparamos o **custo por tonelada**.

| Tipo | Cap. | Custo     | R\$/t  |
|------|------|-----------|--------|
| A    | 10 t | R\$ 800   | R\$ 80 |
| B    | 15 t | R\$ 1.050 | R\$ 70 |
| C    | 25 t | R\$ 1.900 | R\$ 76 |

Portanto, a ordem gulosa é:

$$B \rightarrow C \rightarrow A.$$

**Solução para 100 t**

## Uma regra gulosa

Como as capacidades são diferentes, comparamos o **custo por tonelada**.

| Tipo | Cap. | Custo     | R\$/t  |
|------|------|-----------|--------|
| A    | 10 t | R\$ 800   | R\$ 80 |
| B    | 15 t | R\$ 1.050 | R\$ 70 |
| C    | 25 t | R\$ 1.900 | R\$ 76 |

Portanto, a ordem gulosa é:

$$B \rightarrow C \rightarrow A.$$

### Solução para 100 t

$$x_B = 5, \quad x_C = 1, \quad x_A = 0.$$

$$5(15) + 1(25) = 100 \text{ t.}$$

$$\begin{aligned} C(\mathbf{x}) &= 5(1.050) + 1(1.900) \\ &= \text{R\$ } 7.150. \end{aligned}$$

## Uma regra gulosa

Como as capacidades são diferentes, comparamos o **custo por tonelada**.

| Tipo | Cap. | Custo     | R\$/t  |
|------|------|-----------|--------|
| A    | 10 t | R\$ 800   | R\$ 80 |
| B    | 15 t | R\$ 1.050 | R\$ 70 |
| C    | 25 t | R\$ 1.900 | R\$ 76 |

Portanto, a ordem gulosa é:

$$B \rightarrow C \rightarrow A.$$

### Pergunta

Priorizar o menor custo por tonelada sempre garante a solução de menor custo quando capacidades e disponibilidades são diferentes?

### Solução para 100 t

$$x_B = 5, \quad x_C = 1, \quad x_A = 0.$$

$$5(15) + 1(25) = 100 \text{ t.}$$

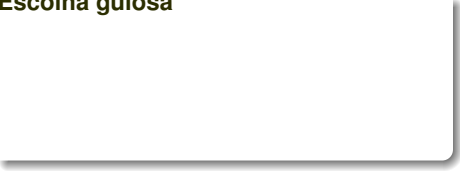
$$\begin{aligned} C(\mathbf{x}) &= 5(1.050) + 1(1.900) \\ &= \text{R\$ } 7.150. \end{aligned}$$

## A regra gulosa sempre retorna a solução ótima?

Considere que precisamos transportar **peelo menos 12 toneladas**.

| Tipo | Cap. | Custo  | R\$/t  |
|------|------|--------|--------|
| A    | 10 t | R\$ 90 | R\$ 9  |
| B    | 6 t  | R\$ 60 | R\$ 10 |

**Escolha gulosa**



## A regra gulosa sempre retorna a solução ótima?

Considere que precisamos transportar **pelo menos 12 toneladas**.

| Tipo | Cap. | Custo  | R\$/t  |
|------|------|--------|--------|
| A    | 10 t | R\$ 90 | R\$ 9  |
| B    | 6 t  | R\$ 60 | R\$ 10 |

### Escolha gulosa

Como A possui o menor custo por tonelada:

$$2A = 20 \text{ t.}$$

$$\text{Custo} = \text{R\$ } 180.$$

## A regra gulosa sempre retorna a solução ótima?

Considere que precisamos transportar **peelo menos 12 toneladas**.

| Tipo | Cap. | Custo  | R\$/t  |
|------|------|--------|--------|
| A    | 10 t | R\$ 90 | R\$ 9  |
| B    | 6 t  | R\$ 60 | R\$ 10 |

### Escolha gulosa

Como A possui o menor custo por tonelada:

$$2A = 20 \text{ t.}$$

$$\text{Custo} = \text{R\$ } 180.$$

### Uma solução melhor

Utilizando dois caminhões do tipo B:

$$2B = 12 \text{ t.}$$

$$\text{Custo} = \text{R\$ } 120.$$

### Comparação

$$\text{R\$ } 120 < \text{R\$ } 180.$$

A solução gulosa transporta 8 t além do necessário e custa R\$ 60 a mais.

## O que acabamos de fazer?

### Heurística

Técnica que procura fornecer uma solução razoável rapidamente, sem garantia formal de otimalidade ou de qualidade da aproximação. Ganha-se eficiência ou simplicidade ao custo de precisão.

### Heurística construtiva

Produz uma solução completa adicionando decisões gradualmente, segundo uma regra de prioridade.

### No exemplo dos caminhões

- 1 Ordenar por custo por tonelada;
- 2 escolher um tipo disponível;
- 3 atualizar a demanda restante;
- 4 repetir até atender à demanda.

A solução obtida será usada como **solução inicial**:

$$\mathbf{x}^{(0)} = (x_A, x_B, x_C).$$

## O que acabamos de fazer?

### Heurística

Técnica que procura fornecer uma solução razoável rapidamente, sem garantia formal de otimalidade ou de qualidade da aproximação. Ganha-se eficiência ou simplicidade ao custo de precisão.

### Heurística construtiva

Produz uma solução completa adicionando decisões gradualmente, segundo uma regra de prioridade.

### Dois papéis das heurísticas

Uma heurística pode **construir** uma solução inicial ou **refinar** uma solução existente. Neste momento, usamos apenas a heurística construtiva.

### No exemplo dos caminhões

- 1 Ordenar por custo por tonelada;
- 2 escolher um tipo disponível;
- 3 atualizar a demanda restante;
- 4 repetir até atender à demanda.

A solução obtida será usada como **solução inicial**:

$$\mathbf{x}^{(0)} = (x_A, x_B, x_C).$$

## Como a mistura entra no mesmo problema?

Até agora, os tipos de caminhão diferiam apenas em **capacidade, custo e disponibilidade**.

Suponha agora que cada tipo seja dedicado ao transporte de uma matéria-prima com propriedades específicas.

| Tipo | Carga           | Capacidade | Proteína | Fibra |
|------|-----------------|------------|----------|-------|
| A    | Matéria-prima A | 10 t       | 24%      | 2%    |
| B    | Matéria-prima B | 15 t       | 10%      | 9%    |
| C    | Matéria-prima C | 25 t       | 18%      | 5%    |

A carga total recebida será utilizada para formar um lote com:

$$\text{proteína} \geq 10\%, \quad 5\% \leq \text{fibra} \leq 10\%.$$

### O que mudou?

Não basta transportar 100 t ao menor custo. A combinação das cargas escolhidas também precisa produzir uma mistura com a qualidade especificada.

## A conexão entre caminhões e mistura

Cada decisão  $x_i$  produz consequências logísticas e de composição.

### Decisão logística

$x_i$  = número de caminhões do tipo  $i$ .

Como cada caminhão possui capacidade  $q_i$ :

$$m_i = q_i x_i,$$

em que  $m_i$  é a massa da matéria-prima  $i$ .

### Contribuição para a qualidade

Seja  $a_{ki}$  o teor do atributo  $k$  na matéria-prima  $i$ .  
Sua contribuição para o lote é:

$$a_{ki} q_i x_i.$$

Por exemplo:

$$a_{ki} = 0,24$$

representa 24% de proteína.

### Elo entre os degraus

A escolha dos caminhões determina simultaneamente a massa de cada matéria-prima e as propriedades da mistura resultante.

## Degrau 3: quantidade, frota e composição

### Massa do lote

Para cada tipo de caminhão:

$q_i x_i$  = massa da matéria-prima  $i$ .

Portanto:

$$\sum_i q_i x_i = Q.$$

### Contribuição do atributo

Se  $a_{ki}$  é o teor do atributo  $k$ :

$a_{ki} q_i x_i$  é sua contribuição para o lote.

### Concentração no lote

$$a_k(\mathbf{x}) = \frac{\sum_i a_{ki} q_i x_i}{Q}.$$

### Limites de qualidade

Se

$$L_k \leq a_k(\mathbf{x}) \leq U_k,$$

então:

$$L_k Q \leq \sum_i a_{ki} q_i x_i \leq U_k Q.$$

### Conclusão

A escolha dos caminhões determina simultaneamente a massa recebida e a qualidade da mistura.

## Degrau 3: quantidade, frota e composição

### Modelo matemático

$$\begin{aligned} \min_{\mathbf{x}} \quad & \sum_i c_i x_i \\ \text{sujeito a} \quad & \sum_i q_i x_i = Q \\ & L_k Q \leq \sum_i a_{ki} q_i x_i \leq U_k Q, \quad \forall k \\ & 0 \leq x_i \leq d_i, \quad \forall i \\ & x_i \in \mathbb{Z}, \quad \forall i. \end{aligned}$$

### Índices

- $i$ : tipo de caminhão e matéria-prima;
- $k$ : atributo de qualidade.

### Variável de decisão

- $x_i$ : número de caminhões do tipo  $i$ .

### Parâmetros

- $c_i$ : custo do caminhão  $i$ ;
- $q_i$ : capacidade do caminhão  $i$ ;
- $d_i$ : disponibilidade do tipo  $i$ ;
- $Q$ : massa total do lote;
- $a_{ki}$ : teor do atributo  $k$  na matéria-prima  $i$ ;
- $L_k, U_k$ : limites do atributo  $k$ .

## A solução mais barata continua viável?

A solução produzida pela regra gulosa foi:

$$x_A = 0, \quad x_B = 5, \quad x_C = 1.$$

## A solução mais barata continua viável?

A solução produzida pela regra gulosa foi:

$$x_A = 0, \quad x_B = 5, \quad x_C = 1.$$

A massa total é atendida:

$$0(10) + 5(15) + 1(25) = 100 \text{ t.}$$

### Proteína

$$\begin{aligned} a_{\text{prot}}(\mathbf{x}) &= \frac{0(10)(0,24) + 5(15)(0,10) + 1(25)(0,18)}{100} \\ &= 0,12 = 12\%. \end{aligned}$$

$$12\% > 10\%.$$

### Fibra

$$\begin{aligned} a_{\text{fibra}}(\mathbf{x}) &= \frac{0(10)(0,02) + 5(15)(0,09) + 1(25)(0,05)}{100} \\ &= 0,08 = 8\%. \end{aligned}$$

$$5\% < 8\% < 10\%.$$

## Degrau 4: a fábrica precisa formar vários lotes

Agora existem lotes  $p = 1, \dots, P$ , possivelmente com massas e especificações diferentes.

### Nova variável

$x_{ip}$  = número de caminhões da matéria-prima  $i$  destinados ao lote  $p$ .

- Cada lote precisa atingir sua própria massa;
- cada lote precisa respeitar seus limites de qualidade;
- os lotes disputam a mesma disponibilidade de caminhões;
- usar um componente em um lote reduz o que resta para os demais.

## Modelo para vários lotes

Seja  $x_{ip}$  o número de caminhões do tipo  $i$  destinados ao lote  $p$ .

$$\begin{aligned} \min_{\mathbf{x}} \quad & \sum_p \sum_i c_i x_{ip} \\ \text{sujeito a} \quad & \sum_i q_i x_{ip} = Q_p, \quad \forall p, \\ & L_{kp} Q_p \leq \sum_i a_{ki} q_i x_{ip} \leq U_{kp} Q_p, \quad \forall k, p, \\ & \sum_p x_{ip} \leq d_i, \quad \forall i, \\ & x_{ip} \in \mathbb{Z}_+, \quad \forall i, p. \end{aligned}$$

### Cada lote

Deve atingir sua massa  $Q_p$  e respeitar seus próprios limites de qualidade  $L_{kp}$  e  $U_{kp}$ .

### Acoplamento

Os lotes compartilham os  $d_i$  caminhões disponíveis de cada tipo e não podem ser otimizados separadamente.

## Duas representações para a mesma solução

### Matriz de contagens $X$

$$X = [x_{ip}] = \begin{bmatrix} 0 & 2 \\ 5 & 0 \\ 1 & 1 \end{bmatrix}.$$

- linha: tipo de caminhão  $i$ ;
- coluna: lote  $p$ ;
- valor: quantidade de caminhões.

### Matriz de posições $S$

$$S = [s_{pj}] = \begin{bmatrix} B & B & B & B & B & C \\ A & A & C & - & - & - \end{bmatrix}.$$

- linha: lote  $p$ ;
- coluna: posição  $j$ ;
- valor: tipo do caminhão na posição.

As duas matrizes descrevem os mesmos caminhões destinados aos mesmos lotes.

## Representação 1: matriz de contagens

Em  $X = [x_{ip}]$ , cada célula informa diretamente quantos caminhões do tipo  $i$  são destinados ao lote  $p$ .

### Exemplo

|   | Lote 1 | Lote 2 |
|---|--------|--------|
| A | 0      | 2      |
| B | 5      | 0      |
| C | 1      | 1      |

$$(x_{A1}, x_{B1}, x_{C1}) = (0, 5, 1).$$

### Informações calculadas a partir de $X$

- massa do lote  $p$ :  $\sum_i q_i x_{ip}$ ;
- uso do tipo  $i$ :  $\sum_p x_{ip}$ ;
- custo:  $\sum_p \sum_i c_i x_{ip}$ ;
- qualidade:  $\frac{\sum_i a_{ki} q_i x_{ip}}{Q_p}$ .

Para o Lote 1:  $0(10) + 5(15) + 1(25) = 100$  t.

## Representação 2: matriz de posições

Em  $S = [s_{pj}]$ , cada linha descreve individualmente os caminhões de um lote:

$$S = \begin{bmatrix} B & B & B & B & B & C \\ A & A & C & - & - & - \end{bmatrix}.$$

### Interpretação

- $s_{pj}$ : tipo na posição  $j$  do lote  $p$ ;
- “-”: posição não utilizada;
- a massa é  $\sum_{j:s_{pj} \neq -} q_{s_{pj}}$ .

### Atenção

Como as capacidades são diferentes, o número de posições não determina a massa. Além disso, a ordem não possui significado físico no exemplo atual.

## Da construção ao refinamento

A heurística construtiva produz uma solução inicial:

$$X^{(0)} = [x_{ip}^{(0)}] \quad \text{ou, equivalentemente,} \quad S^{(0)} = [s_{pj}^{(0)}].$$

### Construção

Parte de uma solução vazia e acrescenta decisões até obter uma solução completa.

### Refinamento

Parte de uma solução completa e realiza modificações para tentar melhorar sua qualidade.

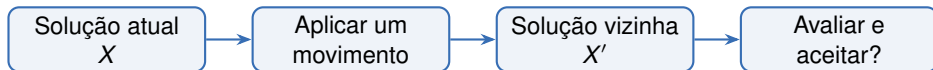
### Próxima pergunta

Quais modificações podem transformar a solução atual em outra solução próxima e potencialmente melhor?

## O que é uma heurística de refinamento?

### Heurística de refinamento

Procedimento que parte de uma solução completa e explora modificações em sua estrutura, procurando reduzir a função objetivo ou melhorar sua factibilidade.



A regra que define a modificação é chamada de **movimento**.

## Movimento, vizinho e vizinhança

### Movimento

Regra que modifica uma solução  $X$  para produzir outra solução  $X'$ .

### Vizinho

$X'$  é vizinha de  $X$  se pode ser obtida pela aplicação de um movimento permitido.

### Vizinhança

$$\mathcal{N}(X) = \{X' : X' \text{ é vizinha de } X\}.$$

É o conjunto de soluções alcançáveis por aquele tipo de movimento.

Uma solução pode ter milhões de vizinhos, mesmo quando o movimento é simples.

## O que é uma estrutura de vizinhança?

### Estrutura de vizinhança

Regra geral que define como uma solução pode ser modificada e, portanto, quais soluções pertencem a  $\mathcal{N}(X)$ .

Ao definir uma estrutura, precisamos indicar:

- quais elementos da solução podem ser alterados;
- como o movimento é realizado;
- quais características são preservadas automaticamente;
- quais restrições precisam ser verificadas;
- se soluções inviáveis podem ser geradas ou aceitas.

Alterar, preservar e verificar são papéis diferentes na definição do movimento.

## Exemplo comum para $N_1$ e $N_2$

Considere a solução inicial:

$$X = \begin{bmatrix} 2 & 1 \\ 2 & 1 \\ 2 & 0 \end{bmatrix},$$

linhas: A, B e C;  
colunas: lotes 1 e 2.

**Lote 1: (2A, 2B, 2C)**

$$Q_1 = 2(10) + 2(15) + 2(25) = 100 \text{ t.}$$

$$P_1 = \frac{2(0,24)(10) + 2(0,10)(15)}{100} + \frac{2(0,18)(25)}{100} = 16,8\%.$$

$$F_1 = \frac{2(0,02)(10) + 2(0,09)(15)}{100} + \frac{2(0,05)(25)}{100} = 5,6\%.$$

**Lote 2: (1A, 1B, 0C)**

$$Q_2 = 1(10) + 1(15) = 25 \text{ t.}$$

$$P_2 = \frac{1(0,24)(10) + 1(0,10)(15)}{25} = 15,6\%.$$

$$F_2 = \frac{1(0,02)(10) + 1(0,09)(15)}{25} = 6,2\%.$$

Uso global: (3A, 3B, 2C), respeitando  $(d_A, d_B, d_C) = (4, 5, 2)$ .

## $N_1$ — passo 1: escolher o movimento

Em  $N_1$ , modificamos a composição de **um único lote**, substituindo um caminhão por outro tipo disponível.

No Lote 1, escolhemos:

$$C \longrightarrow B.$$

Nas contagens do Lote 1:

$$(2, 2, 2) \longrightarrow (2, 3, 1).$$

Na matriz de posições:

$$(A, A, B, B, C, C) \longrightarrow (A, A, B, B, B, C).$$

O número de caminhões é preservado, mas a massa pode mudar porque  $q_B \neq q_C$ .

## $N_1$ — passo 2: atualizar a solução

Somente a primeira coluna de  $X$  é alterada:

$$X = \begin{bmatrix} 2 & 1 \\ 2 & 1 \\ 2 & 0 \end{bmatrix} \longrightarrow X' = \begin{bmatrix} 2 & 1 \\ 3 & 1 \\ 1 & 0 \end{bmatrix}.$$

### Preservado

- composição do Lote 2;
- número de caminhões do Lote 1: 6;
- contagem do tipo A.

### Alterado

- massa do Lote 1: 100 t  $\rightarrow$  90 t;
- uso global: (3, 3, 2)  $\rightarrow$  (3, 4, 1);
- custo total: R\$ 9.350  $\rightarrow$  R\$ 8.500.

## $N_1$ — passo 3: atualizar a qualidade

No Lote 1, antes do movimento:

$$\text{proteína} = 16,8\%, \quad \text{fibra} = 5,6\%.$$

Depois de  $C \rightarrow B$ , a nova massa é 90 t:

$$\text{proteína} = \frac{2(10)(0,24) + 3(15)(0,10) + 1(25)(0,18)}{90} \approx 15,3\%,$$

$$\text{fibra} = \frac{2(10)(0,02) + 3(15)(0,09) + 1(25)(0,05)}{90} \approx 6,3\%.$$

### Resultado de $N_1$

O custo diminuiu R\$ 850 e a qualidade permaneceu nos limites, mas a solução ficou inviável em massa: faltam 10 t no Lote 1.

## $N_2$ — passo 1: escolher a troca

Em  $N_2$ , trocamos conjuntos de caminhões com a mesma massa entre dois lotes. Voltamos à solução inicial  $X$  e escolhemos:

$$\begin{array}{ll} \text{Lote 1: } C & \longrightarrow A + B, \\ \text{Lote 2: } A + B & \longrightarrow C. \end{array}$$

### Lote 1

$$\begin{array}{l} (2, 2, 2) \longrightarrow (3, 3, 1) \\ 100 \text{ t} \longrightarrow 100 \text{ t.} \end{array}$$

### Lote 2

$$\begin{array}{l} (1, 1, 0) \longrightarrow (0, 0, 1) \\ 25 \text{ t} \longrightarrow 25 \text{ t.} \end{array}$$

Os conjuntos trocados possuem a mesma massa:  $q_C = q_A + q_B = 25 \text{ t}$ .

## $N_2$ — passo 2: atualizar a solução

A troca produz:

$$X = \begin{bmatrix} 2 & 1 \\ 2 & 1 \\ 2 & 0 \end{bmatrix} \longrightarrow X'' = \begin{bmatrix} 3 & 0 \\ 3 & 0 \\ 1 & 1 \end{bmatrix}.$$

### Preservado

- massas dos lotes: 100 t e 25 t;
- uso global:  $(3A, 3B, 2C)$ ;
- disponibilidade global;
- custo total: R\$ 9.350;

### Alterado

- composição dos dois lotes;
- qualidade dos dois lotes;
- caminhões no Lote 1:  $6 \rightarrow 7$ ;
- caminhões no Lote 2:  $2 \rightarrow 1$ .

## $N_2$ — passo 3: atualizar a qualidade

|        | Antes    |       | Depois   |       |
|--------|----------|-------|----------|-------|
|        | Proteína | Fibra | Proteína | Fibra |
| Lote 1 | 16,8%    | 5,6%  | 16,2%    | 5,9%  |
| Lote 2 | 15,6%    | 6,2%  | 18,0%    | 5,0%  |

### Por que o custo total não mudou?

O Lote 1 ficou R\$ 50 mais barato e o Lote 2 ficou R\$ 50 mais caro. Os mesmos caminhões apenas mudaram de lote.

### Resultado de $N_2$

A troca preservou massa, disponibilidade e custo total. Como a qualidade também respeitou os limites, a solução permaneceu viável.

## Busca local

Uma busca local aplica repetidamente uma estrutura de vizinhança:

- 1 comece com a solução atual  $X$ ;
- 2 gere ou examine soluções em  $\mathcal{N}(X)$ ;
- 3 escolha uma vizinha  $X'$  com  $f(X') < f(X)$ ;
- 4 faça  $X \leftarrow X'$  e repita;
- 5 pare quando nenhuma melhoria for encontrada.

### Primeira melhoria

Aceita a primeira vizinha melhor encontrada.

### Melhor melhoria

Examina a vizinhança e escolhe a melhor vizinha.

# Busca local — First Improvement

## Ideia

Examine os vizinhos em uma ordem definida e aceite o primeiro que melhorar a solução atual.

---

## Algorithm 1: Heurística First Improvement

---

**Input:** solução  $X$  e estrutura de vizinhança  $\mathcal{N}_k$

**Output:** solução localmente ótima em  $\mathcal{N}_k$

```
1 melhorou ← verdadeiro
2 while melhorou do
3   melhorou ← falso
4   foreach  $Y \in \mathcal{N}_k(X)$ , na ordem de exploração do
5     if  $F(Y) < F(X)$  then
6        $X \leftarrow Y$ 
7       melhorou ← verdadeiro
8       interrompa a exploração da vizinhança
9 return  $X$ 
```

---

Pode economizar avaliações, mas o resultado depende da ordem dos vizinhos.

## Busca local — Best Improvement

### Ideia

Examine toda a vizinhança e escolha a melhor solução admissível.

### Algorithm 2: Heurística Best Improvement

**Input:** solução  $X$  e estrutura de vizinhança  $\mathcal{N}_k$

**Output:** solução localmente ótima em  $\mathcal{N}_k$

```
1 melhorou  $\leftarrow$  verdadeiro
2 while melhorou do
3   melhorou  $\leftarrow$  falso
4    $Y^* \leftarrow X$ 
5   foreach  $Y \in \mathcal{N}_k(X)$ , na ordem de exploração do
6     if  $F(Y) < F(Y^*)$  then
7        $Y^* \leftarrow Y$ 
8   if  $F(Y^*) < F(X)$  then
9      $X \leftarrow Y^*$ 
10    melhorou  $\leftarrow$  verdadeiro
11 return  $X$ 
```

Cada iteração costuma ser mais cara, pois toda a vizinhança é examinada.

## A limitação da busca local

Uma busca local produz uma sequência monotônica de melhorias:

$$f(X^{(0)}) > f(X^{(1)}) > \dots > f(X^*).$$

Ela termina quando não encontra uma solução melhor na vizinhança utilizada:

$$f(X^*) \leq f(Y), \quad \forall Y \in \mathcal{N}(X^*).$$

### Ótimo local

$X^*$  é a melhor solução em sua vizinhança, mas pode não ser a melhor solução de todo o espaço viável.

### Nova pergunta

Como escapar de um ótimo local e explorar outras regiões do espaço de soluções?

## O que é uma metaheurística?

### Metaheurística

Estratégia de alto nível que coordena componentes heurísticos para explorar o espaço de soluções e reduzir a dependência de um único ótimo local.

### Intensificação

Explora detalhadamente uma região promissora.

Pode ser realizada por uma busca local, utilizando:

- uma ou mais vizinhanças;
- First Improvement;
- Best Improvement.

### Diversificação

Conduz a busca para outras regiões do espaço de soluções.

Pode ser realizada por:

- perturbações na solução;
- reinícios da busca;
- aceitação controlada de pioras.

**A busca local explora uma região; a metaheurística também cria mecanismos para sair dela.**

## Busca em Vizinhança Variável (VNS)

A VNS, proposta por Mladenović e Hansen (1997), explora sistematicamente diferentes estruturas de vizinhança.

---

### Algorithm 3: Busca em Vizinhança Variável (VNS)

---

**Input:** solução inicial  $X$ , estruturas de perturbação  $\mathcal{P}_1, \dots, \mathcal{P}_{k_{\max}}$ , estrutura de refinamento  $\mathcal{N}$  e critério de parada

**Output:** melhor solução encontrada

```
1 while o critério de parada não for satisfeito do
2      $k \leftarrow 1$ 
3     while  $k \leq k_{\max}$  do
4          $X' \leftarrow \text{SHAKE}(X, \mathcal{P}_k)$ 
5          $X'' \leftarrow \text{BUSCALOCAL}(X', \mathcal{N})$ 
6         if  $F(X'') < F(X)$  then
7              $X \leftarrow X''$ 
8              $k \leftarrow 1$                                      // aceita e reinicia
9         else
10             $k \leftarrow k + 1$                                // muda a perturbação
11 return  $X$ 
```

---

Perturbar diversifica; a busca local intensifica.

## Variable Neighborhood Descent — VND

O VND realiza o refinamento mudando de vizinhança de maneira determinística.

---

### Algorithm 4: Variable Neighborhood Descent (VND)

---

**Input:** solução  $X$  e vizinhanças  $\mathcal{N}_1, \dots, \mathcal{N}_{\ell_{\max}}$

**Output:** solução localmente ótima nas vizinhanças utilizadas

```
1  $\ell \leftarrow 1$ 
2 while  $\ell \leq \ell_{\max}$  do
3    $X' \leftarrow \text{BUSCALOCAL}(X, \mathcal{N}_{\ell})$ 
4   if  $F(X') < F(X)$  then
5      $X \leftarrow X'$ 
6      $\ell \leftarrow 1$                                      // volta à primeira vizinhança
7   else
8      $\ell \leftarrow \ell + 1$                              // avança para outra vizinhança
9 return  $X$ 
```

---

No exemplo, o VND pode usar  $N_1$  e  $N_2$  como estruturas de refinamento.

## General Variable Neighborhood Search — GVNS

O GVNS é uma VNS na qual o refinamento é realizado pelo VND.

---

### Algorithm 5: General Variable Neighborhood Search (GVNS)

---

**Input:** solução inicial  $X$ , estruturas de perturbação  $\mathcal{P}_1, \dots, \mathcal{P}_{k_{\max}}$ , estruturas de refinamento  $\mathcal{N}_1, \dots, \mathcal{N}_{\ell_{\max}}$  e critério de parada

**Output:** melhor solução encontrada

```
1 while o critério de parada não for satisfeito do
2      $k \leftarrow 1$ 
3     while  $k \leq k_{\max}$  do
4          $X' \leftarrow \text{SHAKE}(X, \mathcal{P}_k)$ 
5          $X'' \leftarrow \text{VND}(X', \mathcal{N}_1, \dots, \mathcal{N}_{\ell_{\max}})$ 
6         if  $F(X'') < F(X)$  then
7              $X \leftarrow X''$ 
8              $k \leftarrow 1$  // aceita e volta à primeira perturbação
9         else
10             $k \leftarrow k + 1$  // tenta outra estrutura de perturbação
11 return  $X$ 
```

---

O Shake diversifica com  $\mathcal{P}_k$ ; o VND intensifica com  $\mathcal{N}_1, \dots, \mathcal{N}_{\ell_{\max}}$ .

## Nem todo movimento produz uma solução viável

Uma substituição ou troca pode violar:

- a disponibilidade de uma matéria-prima;
- um limite mínimo ou máximo de qualidade;
- uma regra de compatibilidade entre material e lote;
- a massa do lote, se a representação não a preservar.

### Decisão algorítmica

Devemos impedir a geração de soluções inviáveis, corrigi-las ou permitir temporariamente sua exploração?

## Estratégia 1: preservar a factibilidade

Antes de realizar o movimento, verifique todas as condições afetadas.

### Para uma substituição $A \rightarrow B$

- ainda há cargas de B disponíveis?
- retirar A respeita alguma exigência mínima?
- B é compatível com o lote?
- a nova composição respeita os limites de qualidade?

### Princípio de projeto

Preserve estruturalmente as restrições fáceis e verifique explicitamente apenas as restrições afetadas pelo movimento.

## Estratégia 2: penalização

Transformamos o problema restrito em uma função ampliada:

$$F(X) = f(X) + \lambda V(X),$$

em que  $V(X) \geq 0$  mede a violação total.

Para  $g_i(X) \leq 0$  e  $h_j(X) = 0$ :

$$V(X) = \sum_i [\max\{0, g_i(X)\}]^2 + \sum_j [h_j(X)]^2.$$

### Escala importa

Violações de massa e de percentuais de qualidade não devem ser somadas sem normalização ou pesos coerentes.

## Estratégia 3: regras de factibilidade

Ao comparar duas soluções:

- 1 se ambas são viáveis, escolha a de menor custo;
- 2 se apenas uma é viável, escolha a viável;
- 3 se ambas são inviáveis, escolha a de menor violação.

## Estratégia 4: reparo

Um operador de reparo transforma uma solução inviável em uma solução viável ou menos inviável.

### Exemplo

- 1 identifique o atributo fora do intervalo;
- 2 procure uma substituição que mova o atributo na direção correta;
- 3 verifique disponibilidade e demais atributos;
- 4 selecione a mudança de menor aumento de custo;
- 5 repita até recuperar a factibilidade.

Reparos incorporam conhecimento do problema, mas podem ser caros e reduzir a diversidade.

## O problema completo possui objetivos conflitantes

Podemos desejar simultaneamente:

$$\min_{\mathbf{x} \in \mathcal{F}} \{f_{\text{custo}}(\mathbf{x}), f_{\text{desvio}}(\mathbf{x}), f_{\text{homogeneidade}}(\mathbf{x})\}.$$

- Materiais baratos podem dificultar o atendimento do alvo;
- lotes muito semelhantes podem exigir componentes mais caros;
- reduzir o pior desvio pode aumentar o desvio médio;
- não existe necessariamente uma solução melhor em tudo.

### Por enquanto

Hoje reconhecemos o conflito. Dominância e soluções de Pareto serão formalizadas posteriormente.

## Literatura básica e complementar

- E. O. Abensur, *Pesquisa operacional para cursos de engenharia de produção*, Editora Blucher, 2018.
- M. J. F. Souza, *Inteligência Computacional para Otimização: meta-heurísticas*, Notas de aula, Departamento de Computação, Universidade Federal de Ouro Preto, Ouro Preto, 2024.
- E.-G. Talbi, *Metaheuristics: From Design to Implementation*, Wiley, 2009.
- J. Dréo, P. Siarry, A. Pétrowski e E. Taillard, *Metaheuristics for Hard Optimization*, Springer, 2006.
- N. Mladenović e P. Hansen, “Variable Neighborhood Search”, *Computers & Operations Research*, 24(11), p. 1097–1100, 1997.
- P. Hansen, N. Mladenović e J. A. Moreno Pérez, “Variable Neighbourhood Search: Methods and Applications”, *Annals of Operations Research*, 175, p. 367–407, 2010.